

Основа: генерация, префилл, вызов инструментов

vlm-recipes · что происходит под остальными тремя гайдами

1. Что модель делает на самом деле

Языковая модель — функция одного вида. На вход последовательность токенов, на выходе распределение вероятностей над следующим токеном:

$$p_{\theta}(\cdot \mid x_1, \dots, x_{t-1}) \in \Delta^{|V|},$$

где $|V|$ — размер словаря, у Qwen порядка 150 тысяч.

Больше она не умеет ничего. Всё остальное — диалог, вызов инструментов, следование инструкции — надстройка над этим единственным действием.

Генерация есть повторное применение: выбрали токен, дописали к входу, применили снова. Отсюда главное следствие, которое стоит принять сразу:

Модель не *отвечает* на вопрос. Она *продолжает* текст. Ощущение ответа возникает потому, что текст оформлен так, что естественным продолжением оказывается реплика ассистента.

2. Диалог как одна строка

Роль «пользователь» и «ассистент» для модели не существует. Существуют токены. Список реплик превращается в одну строку по правилу, которое называется chat template и хранится в токенизаторе модели.

У Qwen это выглядит так:

```
<|im_start|>system
Ты помогаешь студенту.<|im_end|>
<|im_start|>user
Сформулируй гипотезу<|im_end|>
<|im_start|>assistant
```

Обратите внимание на последнюю строку: она открыта. Реплика ассистента начата и не закончена, и модели остаётся её продолжить — ровно то, что она умеет. Служебные токены `<|im_start|>` и `<|im_end|>` — обычные элементы словаря, просто зарезервированные под разметку.

Дописывает эту открывающую строку флаг `add_generation_prompt=True`. Без него текст заканчивается на `<|im_end|>`, и модель с равным успехом может начать новую реплику пользователя.

Точный формат различается между семействами, поэтому смотреть надо свой:

```
print(processor.apply_chat_template(messages, tokenize=False,
                                    add_generation_prompt=True))
```

3. Префилл

Приём следует прямо из устройства генерации. Раз модель продолжает текст, начало ответа можно задать самому — тогда она продолжит уже написанное.

Технически: добавляем реплику ассистента с нужным началом и просим шаблон не закрывать её.

```
messages = [
    {"role": "user", "content": "Сформулируй гипотезу"},
    {"role": "assistant", "content": "Уточняющий вопрос:"},
]
```

```
prompt = processor.apply_chat_template(
    messages, tokenize=False, continue_final_message=True
)
```

Получается строка, оканчивающаяся на ...assistant\n Уточняющий вопрос: — и генерация идёт с этой точки.

Флаг `continue_final_message=True` здесь обязателен, и одного `add_generation_prompt=False` недостаточно. Шаблон отрисовывает каждую реплику как `<|im_start|>роль ... <|im_end|>`, то есть закрыл бы и вашу: получилось бы Уточняющий вопрос:<|im_end|> — реплика завершена, продолжать нечего, префилл не работал.

Проверяется это на своей модели за секунду:

```
for kw in ({"add_generation_prompt": False}, {"continue_final_message": True}):
    print(kw, repr(proc.apply_chat_template(messages, tokenize=False, **kw))[-60:])
```

3.1. Почему это сильнее инструкции

Инструкция «задай уточняющий вопрос» попадает в условие $x_{<t}$ наравне с текстом пользователя и конкурирует с ним: модель взвешивает всё содержимое контекста и *решает*, следовать ли ей.

Префилл не участвует в этом соревновании. Он меняет саму точку, из которой разворачивается авторегрессия:

$$y \sim p_{\theta}(\cdot | x), \quad \text{против} \quad y \sim p_{\theta}(\cdot | x, y_1, \dots, y_k),$$

где $y_1, \dots, y_k$ заданы вами и уже не подлежат пересмотру. Модель не решает, задавать ли вопрос, — она его уже задала и достраивает начатое.

Отсюда область применения: префилл идеален там, где форму ответа выбирает код, а содержание — модель. Состояние документа известно программе, она и решает, каким должно быть начало реплики.

3.2. Чем это отличается от системного промпта

Системный промпт работает не потому, что «системный», а потому что при инструктивном дообучении в данных систематически шло: описание поведения в системной части, затем соответствующая ему реплика ассистента. Модель выучила эту корреляцию. Механизм статистический — оттого систему и можно перебить последующим текстом.

Префилл статистическим не является вовсе. Он не свидетельство, которое модель взвешивает, а часть самой последовательности:

системный промпт	свидетельство, которое модель взвешивает
префилл	часть самой последовательности

Первое можно перевесить другим свидетельством. Второе перевешивать нечем: оно не в споре, оно уже случилось.

3.3. Граница действия

Префилл фиксирует первые k токенов и ровно на этом заканчивается — дальше модель снова свободна. Начав «Уточняющий вопрос:», она обязана продолжить с этой точки, но через десяток токенов может задать вопрос и тут же сама на него ответить.

Гарантируется начало, а не весь ответ. Поэтому в лестнице приёмов префилл стоит рядом со структурированным декодированием, а не вместо него: первое задаёт точку старта, второе ограничивает каждый шаг.

4. Как получается вызов инструмента

Здесь чаще всего возникает неверная картина, поэтому скажем прямо:

Модель ничего не вызывает. Она порождает строку определённого вида. Разбирает эту строку, выполняет функцию и возвращает результат **ваш код**.

4.1. Что видит модель

Описания доступных функций подставляются в промпт — обычно в системную часть, в формате, на котором модель обучалась. Передаются они отдельным аргументом, а не текстом:

```
tools = [{
    "type": "function",
    "function": {
        "name": "read_document",
        "description": "Текст раздела работы",
        "parameters": {
            "type": "object",
            "properties": {"section_id": {"type": "string"}},
            "required": ["section_id"],
        },
    },
}]
processor.apply_chat_template(messages, tools=tools, tokenize=False,
                              add_generation_prompt=True)
```

Шаблон сам развернёт это в текст. Если у модели в `chat_template` нет ветки по `tools`, вызов инструментов у неё не обучен — просить можно, но формат придётся разбирать самому и надёжность будет ниже.

4.2. Что порождает модель

Обычный текст, в котором есть размеченный фрагмент. Единого формата нет — он задаётся шаблоном конкретной модели. Два распространённых:

<pre><tool_call> {"name": "read_document", "arguments": {"section_id": "3.2"}} </tool_call></pre>	<pre><tool_call> <function=read_document> <parameter=section_id> 3.2 </parameter> </function> </tool_call></pre>
--	--

Слева JSON, справа вложенные XML-теги; Qwen3.5 использует правый. Какой ждёт ваша модель, написано прямо в системной части промпта после подстановки `tools` — и это же означает, что обучающие данные должны быть в том же формате, иначе вы учите модель одному, а разбираете другое.

Никакого особого механизма за этим нет. Модель обучена: увидев описания функций и запрос, для которого нужен вызов, продолжать текст такой конструкцией. Это тот же авторегрессионный процесс, что порождает любое другое предложение.

4.3. Полный цикл

```
messages = [{"role": "user", "content": "Прочитай раздел 3.2"}]

while True:
    text = generate(messages, tools=tools)    # шаг модели
```

```
calls = parse_tool_calls(text)          # разбор строки
if not calls:
    break                               # модель ответила словами
messages.append({"role": "assistant", "content": text})
for call in calls:
    result = TOOLS[call["name"]](**call["arguments"]) # ваш код
    messages.append({"role": "tool", "content": result})
```

Результат возвращается в диалог как реплика роли `tool`, и следующий шаг модели видит его наравне с остальным контекстом. Цикл повторяется, пока модель не перестанет порождать вызовы.

Отсюда два практических следствия. Первое: остановка — это тоже поведение модели, и ему нужно учить отдельно, иначе цикл не завершится. Второе: результат инструмента приходит извне, и обучаться предсказывать его нельзя — модель начнёт сочинять содержимое документов вместо того, чтобы их запрашивать. В коллаторе эти позиции маскируются.

5. Префилл и вызов инструмента вместе

Приёмы складываются. Если нужно гарантировать, что модель вызовет инструмент, начало вызова задаётся префиллом:

```
{"role": "assistant", "content": '<tool_call>\n{"name": ""}'
```

Дальше модели остаётся продолжить именем функции — вызывать или не вызывать она уже не решает. Дешёвая замена структурированному декодированию, когда его нет под рукой.

Тот же приём наоборот, для форсирования обычного ответа: префилл непустым текстом закрывает путь к `<tool_call>` на первом же шаге.

6. Рассуждающие модели и блок `<think>`

У рассуждающих моделей шаблон открывает блок размышления сразу после служебных токенов ассистента — промпт кончается так:

```
<|im_start|>assistant
<think>
```

Модель сначала рассуждает внутри блока, затем закрывает его тегом `</think>` и только потом пишет ответ. Из этого следуют четыре вещи.

Префилл ответа должен сначала закрыть блок. Дописав «Уточняющий вопрос:» прямо после `<think>`, вы префиллите *рассуждение*, а не ответ. Чтобы префиллить ответ:

```
prompt += "\n</think>\n\nУточняющий вопрос:"
```

Пустой блок `<think>\n\n</think>` — это не мусор, а сигнал «рассуждать не нужно». Шаблон сам подставляет его в завершённые реплики.

В истории диалога рассуждение выбрасывается. Оно нужно для текущего шага и на следующем только съест контекст и собьёт модель. Шаблоны Qwen вырезают его сами из всех реплик, кроме последней. Рассуждение живёт один ход.

В обучающих данных без рассуждений шаблон вставит пустой блок. Если этот блок попадёт в градиент, модель усвоит «размышлять не нужно», и дообучение под правила поведения заодно отучит её думать. Поэтому коллатор маскирует `<think>...</think>`: градиент не идёт ни за рассуждение, ни против него.

Лимит токенов уходит в рассуждение. При `max_new_tokens=160` модель успевает только начать думать — по-английски, независимо от языка вопроса, — и ответ не начинается вовсе. Это не «модель ответила на английском», а обрезанное рассуждение. Штатный выключатель — аргумент шаблона:

```
processor.apply_chat_template(messages, add_generation_prompt=True,
                             enable_thinking=False)
```

Шаблон дописывает пустой блок сам, и промпт кончается ровно тем, что стоит перед ответом в обучающих данных: инференс и обучение видят один и тот же префикс. В ноутбуках и замерах рассуждение выключено именно так. Второй способ — префикс `</think>` после шаблона — показан в `tools/agent_loop.py`.

7. Выбор токена

Распределение получено — остаётся выбрать из него токен.

Способ	Когда
жадный, <code>temperature=0</code>	разбор структуры, вызовы инструментов, распознавание текста — везде, где ответ должен быть воспроизводим
<code>temperature 0.6–0.8</code>	свободный диалог; ниже ответы однообразны, выше теряется связность
<code>top_p 0.9</code>	отсекает хвост распределения, обычно вместе с температурой

Для агентского цикла температура нулевая по причине более важной, чем воспроизводимость: разбор вызова ломается от одного лишнего символа, и случайность здесь работает исключительно против вас.

8. Куда это ведёт

Всё, что разбирается в остальных трёх гайдах, стоит на этой основе.

Дообучение (02-sft-math) меняет распределение $p_\theta(\cdot | x)$ — какие продолжения модель считает вероятными. Маскирование определяет, за какие позиции она получает градиент.

Выравнивание (03-alignment) сравнивает два продолжения одного и того же x и сдвигает распределение в пользу одного из них.

Векторы управления вмешиваются между входом и распределением, сдвигая активации на промежуточном слое.

Промпт, префикс и грамматика при декодировании не трогают θ вовсе: первый меняет условие, второй — точку старта, третья — множество допустимых токенов на каждом шаге. Поэтому они бесплатны и поэтому пробовать их следует раньше обучения.